



Departamento de Lenguajes y
Sistemas Informáticos

Implementación de Servicios RESTful (II)

Integración

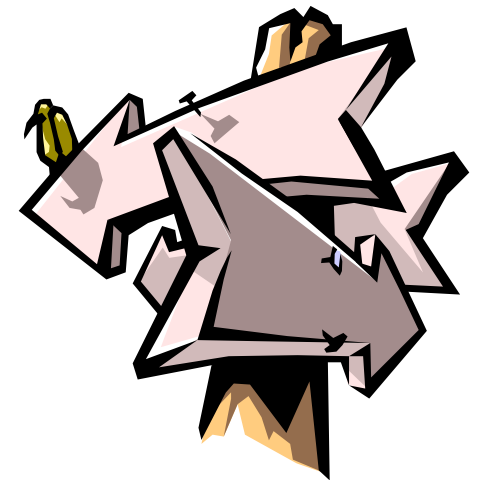


Arquitectura e Integración del Software

2025/2026

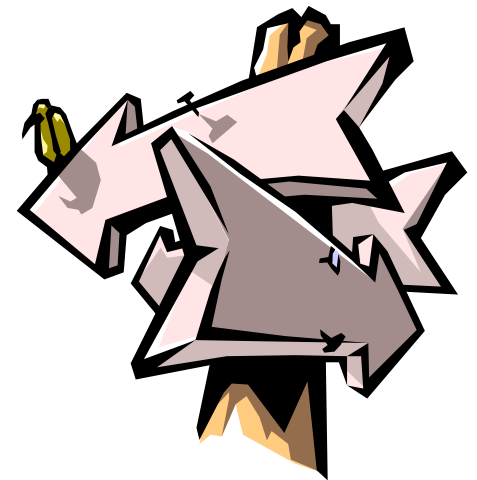
Índice

- Introducción
- Conexión a base de datos
- Gestión de excepciones
 - Validación de parámetros de entrada
 - Creación de excepciones personalizadas



Índice

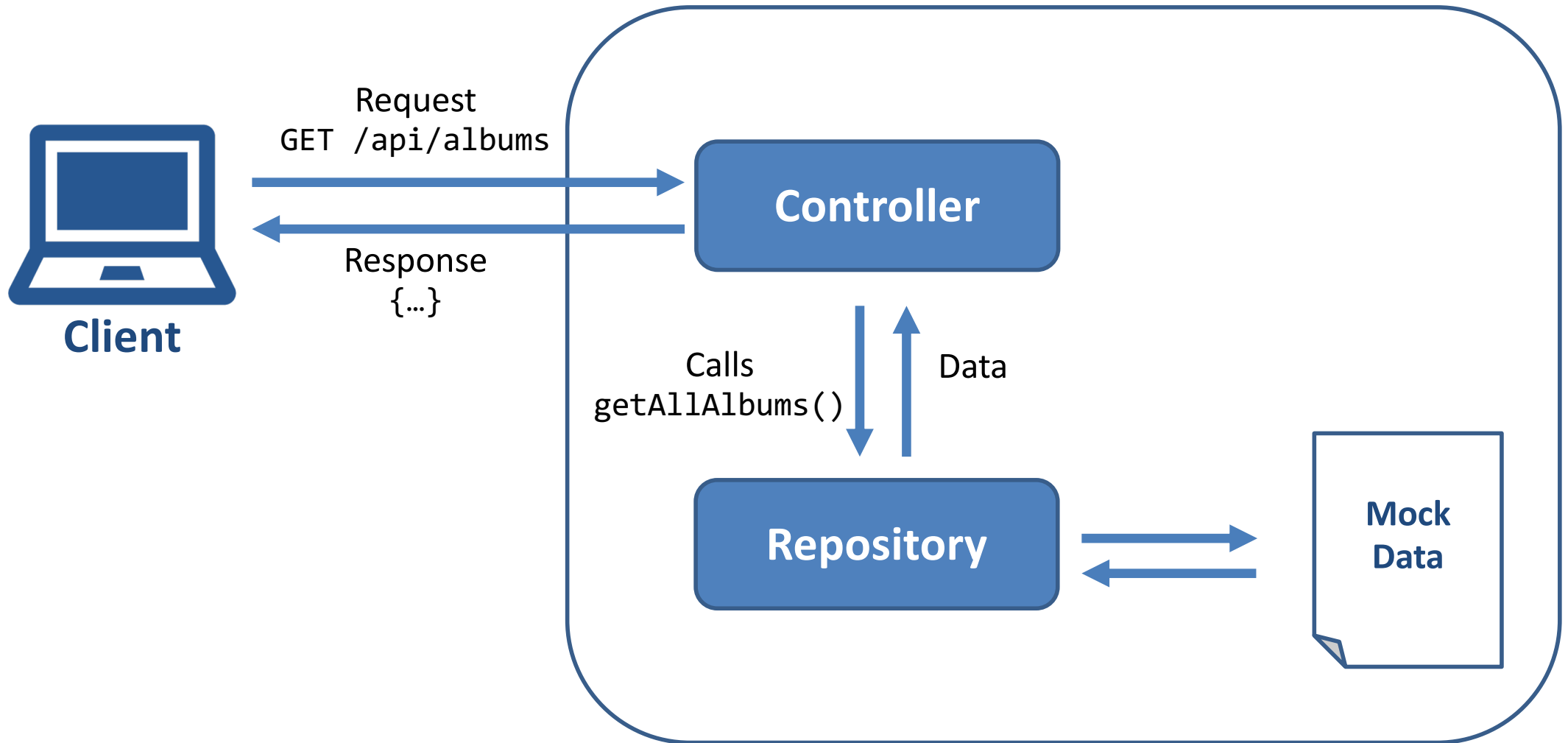
- **Introducción**
- Conexión a base de datos
- Gestión de excepciones
 - Validación de parámetros de entrada
 - Creación de excepciones personalizadas



Introducción

Estructura del proyecto de Spring Boot

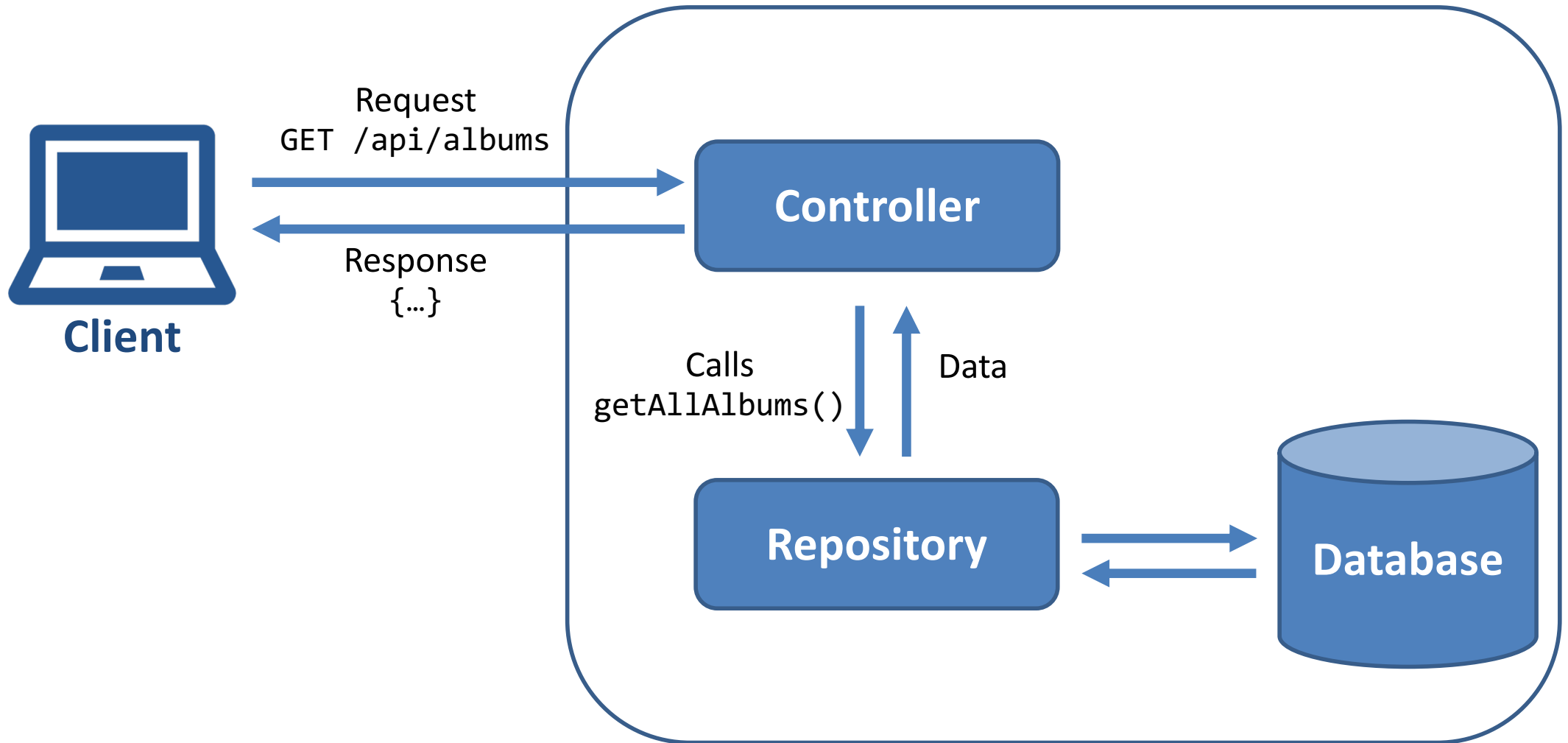
Server (Spring Boot API)



Introducción

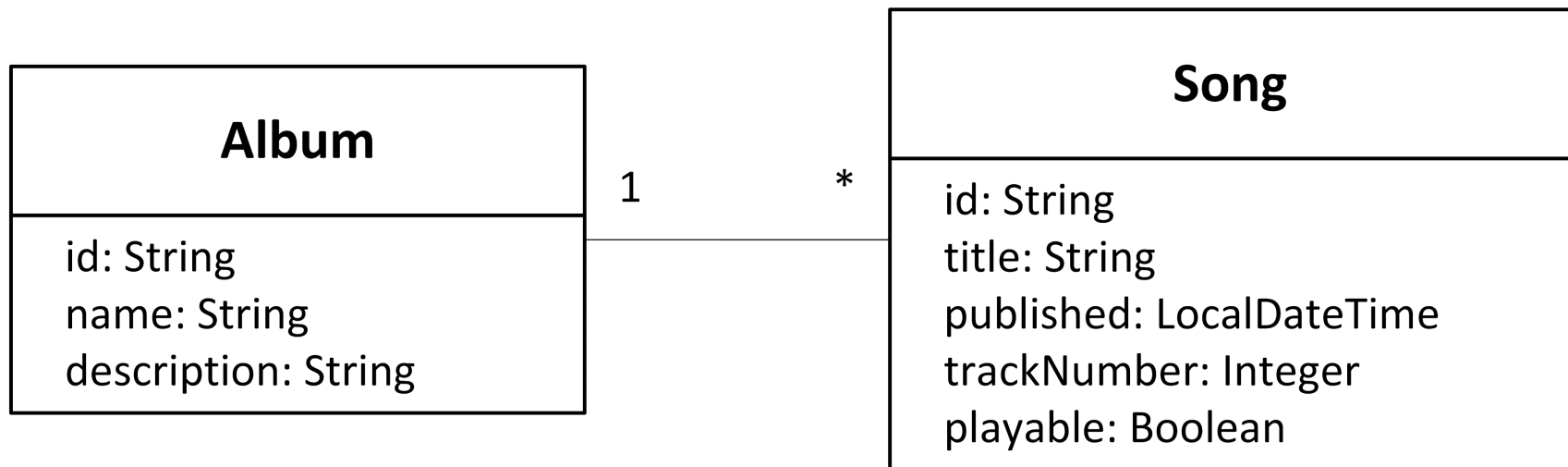
Estructura del proyecto de Spring Boot

Server (Spring Boot API)



Introducción

Modelo de datos



Introducción

Operaciones sobre los recursos

Recurso Album

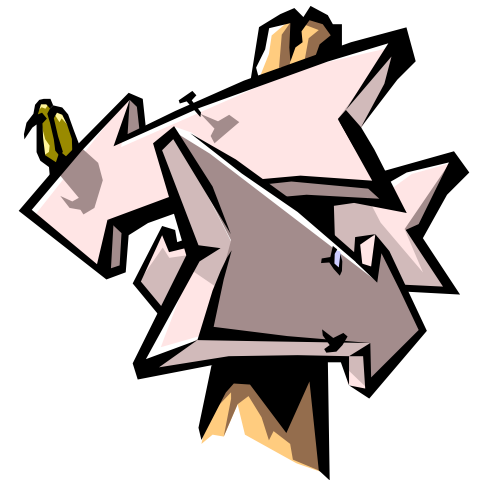
HTTP	URI	Descripción
GET	/api/albums	Lista todos los álbumes de la aplicación
GET	/api/albums/{albumId}	Devuelve el álbum con id=albumId
POST	/api/albums	Crea un nuevo album
PUT	/api/albums/{albumId}	Actualiza el álbum con id=albumId
DELETE	/api/albums/{albumId}	Elimina el álbum con id=albumId

Recurso Song

HTTP	URI	Descripción
GET	/api/albums/{albumId}/songs	Lista todas las canciones del álbum con id=albumId
GET	/api/songs/{songId}	Devuelve la canción con id=songId
POST	/api/albums/{albumId}/songs	Crea una nueva canción en el álbum con id=albumId
PUT	/api/songs/{songId}	Actualiza la canción con id=songId
DELETE	/api/songs/{songId}	Elimina la canción con id=songId

Índice

- Introducción
- **Conexión a base de datos**
- Gestión de excepciones
 - Validación de parámetros de entrada
 - Creación de excepciones personalizadas



Conexión a base de datos

Modificar archivo `src/main/resources/application.properties`

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto= update

spring.h2.console.enabled=true
# default path: h2-console
spring.h2.console.path=/h2-ui
```

Conexión a base de datos

Modificar archivo `src/main/resources/application.properties`

- **spring.datasource.url:** Su valor es `jdbc:h2:mem:[database-name]` si queremos crear una base de datos en memoria y `jdbc:h2:file:[path/database-name]` para base de datos en disco.
- **spring.datasource.driverClassName:** La clase del controlador JDBC (Java Database connectivity) que se usará para conectarse a la base de datos H2.
- **spring.datasource.username** y **spring.datasource.password:** Usuario y contraseña de la base de datos.
- **spring.jpa.show-sql:** Indica si Hibernate mostrará las consultas SQL generadas en la consola de registro.
- **Spring.jpa.properties.hibernate.dialect:** Dialecto SQL específico de la base de datos. En este caso, H2.
- **spring.jpa.hibernate.ddl-auto:** Utilizado para inicializar la base de datos. Su valor es “update” para que las tablas se creen automáticamente de acuerdo a la definición del modelo de datos.
- **spring.h2.console.enabled=true:** Permite a Spring iniciar la herramienta de administración de la base de datos H2 para poder acceder en el navegador: `localhost:8080/h2-console`
- **spring.h2.console.path=/h2-ui:** URL de la consola de H2: `localhost:8080/h2-ui`

Conexión a base de datos

Modificar modelos de datos

Indicamos que se trata de una entidad de la DB

`@Entity`

Especificamos el nombre de la tabla

`@Table(name="albums")`

Indicamos la propiedad que se usa como id

`@Id`

`@GeneratedValue(strategy = GenerationType.IDENTITY)`

`private long id;`

Cambiamos el tipo de dato del id a long

`@Column(name="name")`

`private String name;`

Especificamos el nombre de cada columna de la DB

`@Column(name="description")`

`private String description;`

Indicar si existen relaciones del tipo OneToMany, ManyToOne o OneToOne

`@OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY, orphanRemoval = true)`

`@JoinColumn(name="album_id")`

`private List<Song> songs;`

Creamos un constructor vacío (requisito de Spring)

`public Album() {
}`

Borramos el id del constructor

`public Album(String name, String description) {
 this.name = name;
 this.description = description;
 this.songs = new ArrayList<>();
}`

Conexión a base de datos

Creación de repositorios

Indicamos que se trata de un repositorio

```
@Repository  
public interface AlbumRepository extends JpaRepository<Album, Long> {  
}
```

Utilizamos una interfaz en lugar de una clase

La interfaz extiende a JpaRepository

Datatype de la entidad

Datatype del ID

Conexión a base de datos

Actualizar controladores para que hagan uso del repositorio (1)

Inyectamos el repositorio como dependencia

```
@RestController
@RequestMapping("/api/albums")
public class AlbumController {

    @Autowired
    AlbumRepository repository;

    // GET http://localhost:8080/api/albums
    @GetMapping
    public List<Album> findAll() { return repository.findAll(); }
```

Llamamos a los métodos del repositorio desde el controlador

Conexión a base de datos

Actualizar controladores para que hagan uso del repositorio (2)

- Para obtener un solo álbum dado su id.

```
// GET http://localhost:8080/api/albums/{id}
@GetMapping("/{id}")
public Album findOne(@PathVariable long id) {
    Optional<Album> album = repository.findById(id);
    return album.get();
}
```

- Para crear un nuevo álbum.

```
// POST http://localhost:8080/api/albums
@ResponseStatus(HttpStatus.CREATED)
@PostMapping
public Album create(@Valid @RequestBody Album album) {
    Album _album = repository
        .save(new Album(album.getName(), album.getDescription()));
    return _album;
}
```

Conexión a base de datos

Actualizar controladores para que hagan uso del repositorio (3)

- Actualizar un álbum existente.

```
// PUT http://localhost:8080/api/albums/{id}
@ResponseStatus(HttpStatus.NO_CONTENT)
@PutMapping("/{id}")
public void update(@Valid @RequestBody Album updatedAlbum, @PathVariable long id) {
    Optional<Album> albumData = repository.findById(id);

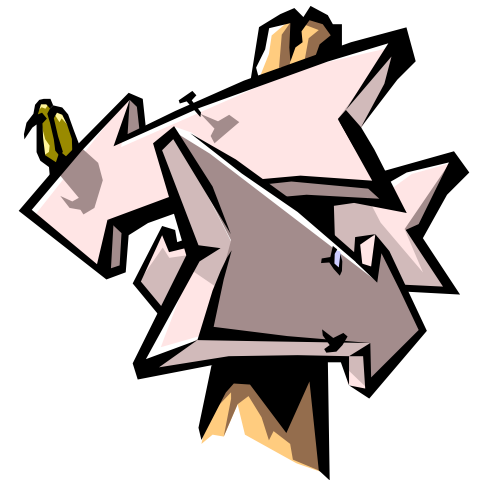
    Album _album = albumData.get();
    _album.setName(updatedAlbum.getName());
    _album.setDescription(updatedAlbum.getDescription());
    repository.save(_album);
}
```

- Eliminar un álbum.

```
// DELETE http://localhost:8080/api/albums/{id}
@ResponseStatus(HttpStatus.NO_CONTENT)
@DeleteMapping("/{id}")
public void delete(@PathVariable long id) {
    if (repository.existsById(id)) {
        repository.deleteById(id);
    }
}
```

Índice

- Introducción
- Conexión a base de datos
- **Gestión de excepciones**
 - **Validación de parámetros de entrada**
 - Creación de excepciones personalizadas



Gestión de excepciones

Validación de parámetros de entrada (1)

```
@Column(name="title")
@NotEmpty(message="Song title is required")
private String title;

@Column(name="published")
@NotNull(message = "Publication date cannot be null")
private LocalDateTime published;

@Column(name="trackNumber")
@Min(value = 1, message = "The track number must be positive")
private Integer trackNumber;
```

- **Spring Boot 2.X:** <https://javaee.github.io/javaee-spec/javadocs/javax/validation/constraints/package-summary.html>
- **Spring Boot 3.X:** <https://jakarta.ee/specifications/bean-validation/3.0/apidocs/jakarta/validation/constraints/package-summary>

Gestión de excepciones

Validación de parámetros de entrada (2)

```
@ControllerAdvice
public class GlobalExceptionHandler {

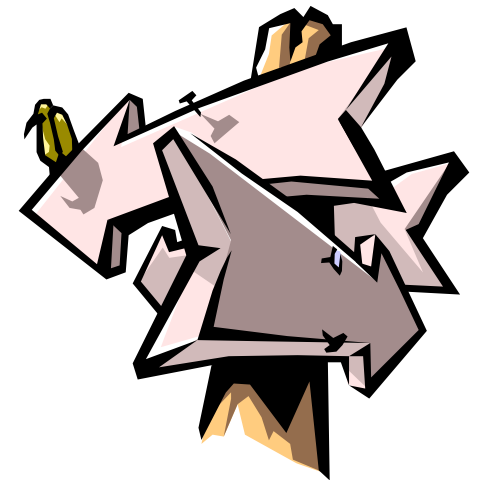
    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseBody
    public ResponseEntity<Map<String, List<String>>> handleValidationException(MethodArgumentNotValidException ex) {
        List<FieldError> fieldErrors = ex.getBindingResult().getFieldErrors();
        List<String> errors = fieldErrors.stream()
            .map(FieldError::getDefaultMessage)
            .collect(Collectors.toList());

        Map<String, List<String>> res = new HashMap<>();
        res.put("errors", errors);

        return new ResponseEntity<>(res, HttpStatus.BAD_REQUEST);
    }
}
```

Índice

- Introducción
- Conexión a base de datos
- **Gestión de excepciones**
 - Validación de parámetros de entrada
 - **Creación de excepciones personalizadas**



Gestión de excepciones

Creación de excepciones personalizadas

- Definimos una nueva excepción dentro del paquete “exception”.

```
@ResponseStatus(code= HttpStatus.NOT_FOUND, reason = "Album not found")
public class AlbumNotFoundException extends Exception {
}
```

- Actualizamos el controlador para que contemple estas excepciones.

```
// GET http://localhost:8080/api/albums/{id}
@GetMapping("/{id}")
public Album findOne(@PathVariable long id) throws AlbumNotFoundException {
    Optional<Album> album = repository.findById(id);
    if(!album.isPresent()) {
        throw new AlbumNotFoundException();
    }
    return album.get();
}
```

Disclaimer and Terms of Use

All material displayed on this presentation is for teaching and personal use only.

Many of the images that have been used in the presentation are Royalty Free images taken from <http://www.everystockphoto.com/>. Other images have been sourced directly from the Public domain, from where in most cases it is unclear whether copyright has been explicitly claimed. Our intention is not to infringe any artist's copyright, whether written or visual. We do not claim ownership of any image that has been freely obtained from the public domain. In the event that we have freely obtained an image or quotation that has been placed in the public domain and in doing so have inadvertently used a copyrighted image without the copyright holder's express permission we ask that the copyright holder writes to us directly, upon which we will contact the copyright holder to request full written permission to use the quote or images.